

List.Perm — Lean 4 / Mathlib Cheatsheet

Notation: $l_1 \sim l_2 \equiv \text{List.Perm } l_1 \ l_2$

Source: `Init.Data.List.Perm` (Lean 4.30)

`List.Perm l1 l2` means `l1` and `l2` contain the same elements with the same multiplicities, possibly in a different order. It is an **equivalence relation** (reflexive, symmetric, transitive) and is preserved by most list operations.

Equivalence Relation

`Perm.refl` every list is $\sim$ itself

$l \sim l$

`Perm.symm` flip direction

$l_1 \sim l_2 \rightarrow l_2 \sim l_1$

`Perm.trans` chain two $\sim$ s

$l_1 \sim l_2 \rightarrow l_2 \sim l_3 \rightarrow l_1 \sim l_3$

`perm_comm` symmetric, iff form

$l_1 \sim l_2 \leftrightarrow l_2 \sim l_1$

`Perm.of_eq` equality implies $\sim$

$l_1 = l_2 \rightarrow l_1 \sim l_2$

Constructors

`Perm.nil` `[]` is $\sim$ itself

$[] \sim []$

`Perm.cons` prepend same element to both

$l_1 \sim l_2 \rightarrow a :: l_1 \sim a :: l_2$

`Perm.swap` swap first two elements

$a :: b :: l \sim b :: a :: l$

Base Cases / Special Lists

`perm_nil` only `[]` is $\sim$

$l \sim [] \leftrightarrow l = []$

`nil_perm` `[]` is $\sim$ only `[]`

$[] \sim l \leftrightarrow l = []$

`perm_singleton` only `[a]` is $\sim$ `[a]`

$l \sim [a] \leftrightarrow l = [a]$

`singleton_perm` `[a]` is $\sim$ only itself

$[a] \sim l \leftrightarrow [a] = l$

`singleton_perm_singleton` $\sim$ iff elements are equal

$[a] \sim [b] \leftrightarrow a = b$

`perm_replicate` $\sim$ `n` copies of `a` iff `l` is exactly those copies

$l \sim \text{replicate } n \ a \leftrightarrow \dots$

`not_perm_nil_cons` `[]` is never $\sim$ non-empty

$\neg [] \sim a :: l$

Membership & Subset

`Perm.mem_iff` membership preserved

$l_1 \sim l_2 \rightarrow (a \in l_1 \leftrightarrow a \in l_2)$

`Perm.subset` implies subset

$l_1 \sim l_2 \rightarrow l_1 \subseteq l_2$

`Perm.contains_eq` Bool membership preserved

$l_1 \sim l_2 \rightarrow l_1.\text{contains } a = l_2.\text{contains } a$

Length & Size

`Perm.length_eq` length preserved

$l_1 \sim l_2 \rightarrow l_1.\text{length} = l_2.\text{length}$

`Perm.isEmpty_eq` emptiness preserved

$l_1 \sim l_2 \rightarrow l_1.\text{isEmpty} = l_2.\text{isEmpty}$

`Perm.sizeOf_eq_sizeOf` `sizeOf` preserved

$l_1 \sim l_2 \rightarrow \text{sizeOf } l_1 = \text{sizeOf } l_2$

Append & Structural

`Perm.append` both halves independently

$l_1 \sim l_2 \rightarrow t_1 \sim t_2 \rightarrow l_1 ++ t_1 \sim l_2 ++ t_2$

`Perm.append_left` right half only

$t_1 \sim t_2 \rightarrow l ++ t_1 \sim l ++ t_2$

Perm.append_right left half only

$l_1 \sim l_2 \rightarrow l_1 ++ t \sim l_2 ++ t$

Perm.append_cons both segments around a shared element

$p_1 : l_1 \sim l_2 \rightarrow p_2 : r_1 \sim r_2 \rightarrow l_1 ++ a :: r_1 \sim l_2 ++ a :: r_2$

perm_append_comm ++ is commutative up to ~

$l_1 ++ l_2 \sim l_2 ++ l_1$

perm_append_comm_assoc swap first two segments

$l_1 ++ l_2 ++ l_3 \sim l_2 ++ l_1 ++ l_3$

Reordering / Middle

perm_middle move middle element to front

$l_1 ++ a :: l_2 \sim a :: (l_1 ++ l_2)$

perm_append_singleton move trailing element to front

$l ++ [a] \sim a :: l$

concat_perm concat a equals prepending a

$l.concat\ a \sim a :: l$

reverse_perm reverse gives ~

$l.reverse \sim l$

perm_cons_append_cons insert a at the split point

$l \sim l_1 ++ l_2 \rightarrow a :: l \sim l_1 ++ a :: l_2$

perm_insertIdx insertIdx i x l ~ x::l for any valid i

$perm\ (insertIdx\ i\ x\ l)\ (x :: l)$

cons_append_cons_perm swap two cons-headed segments

$a :: as ++ b :: bs \sim b :: bs ++ a :: as$

Cons Cancellation

Perm.cons_inv cancel identical heads

$a :: l_1 \sim a :: l_2 \rightarrow l_1 \sim l_2$

perm_cons cancel equal heads (iff)

$a :: l_1 \sim a :: l_2 \leftrightarrow l_1 \sim l_2$

perm_append_left_iff cancel shared left prefix

$l ++ l_1 \sim l ++ l_2 \leftrightarrow l_1 \sim l_2$

perm_append_right_iff cancel shared right suffix

$l_1 ++ l \sim l_2 ++ l \leftrightarrow l_1 \sim l_2$

perm_inv_core cancel shared middle element

$l_1 ++ a :: r_1 \sim l_2 ++ a :: r_2 \rightarrow l_1 ++ r_1 \sim l_2 ++ r_2$

Erase & Insert

perm_cons_erase erase a member and prepend it

$a \in l \rightarrow l \sim a :: l.erase\ a$

cons_perm_iff_erase $a \in l_2$ and tails ~ after erasing a

$a :: l_1 \sim l_2 \leftrightarrow a \in l_2 \wedge l_1 \sim l_2.erase\ a$

Perm.erase erase same element preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{erase } a \sim l_2.\text{erase } a$

Perm.eraseP erase first match preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{eraseP } f \sim l_2.\text{eraseP } f$

Map / Filter / FlatMap

Perm.map map with same f preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{map } f \sim l_2.\text{map } f$

Perm.filter filter with same p preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{filter } p \sim l_2.\text{filter } p$

Perm.filterMap filterMap with same f preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{filterMap } f \sim l_2.\text{filterMap } f$

Perm.flatMap_right flatMap with same f preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{flatMap } f \sim l_2.\text{flatMap } f$

filter_append_perm filter p and $\neg p$, re-appended, recover l
 $\text{filter } p \ l \ ++ \ \text{filter } (!p \ \cdot) \ l \sim l$

Counting

Perm.count_eq element counts preserved
 $l_1 \sim l_2 \rightarrow \text{count } a \ l_1 = \text{count } a \ l_2$

Perm.countP_eq predicate counts preserved
 $l_1 \sim l_2 \rightarrow l_1.\text{countP } p = l_2.\text{countP } p$

perm_iff_count $\sim$ iff all element counts match
 $l_1 \sim l_2 \leftrightarrow \forall a, \text{count } a \ l_1 = \text{count } a \ l_2$

isPerm_iff decidable test equivalent to $\sim$
 $l_1.\text{isPerm } l_2 \leftrightarrow l_1 \sim l_2$

Sorted Lists & Pairwise

Perm.pairwise_iff pairwise R preserved (R must be symmetric)
 $l_1 \sim l_2 \rightarrow (\text{Pairwise } R \ l_1 \leftrightarrow \text{Pairwise } R \ l_2)$

Perm.pairwise pairwise R transfers
 $l \sim l' \rightarrow \text{Pairwise } R \ l \rightarrow \text{Pairwise } R \ l'$

Perm.nodup Nodup transfers
 $l \sim l' \rightarrow \text{Nodup } l \rightarrow \text{Nodup } l'$

Perm.nodup_iff Nodup equivalent across $\sim$
 $l \sim l' \rightarrow (\text{Nodup } l \leftrightarrow \text{Nodup } l')$

Perm.eq_of_pairwise pairwise irrefl R forces equality
 $l_1 \sim l_2 \rightarrow \text{Pairwise } R \ l_1 \rightarrow \text{Pairwise } R \ l_2 \rightarrow l_1 = l_2$

Sublist / Sublists

exists_perm_sublist sublist lifts across $\sim$ of superlist
 $l_1 <+ l_2 \rightarrow l_2 \sim l_2' \rightarrow \exists l_1', l_1' \sim l_1 \wedge l_1' <+ l_2'$

Sublist.exists_perm_append l_1 is a complementary segment
 $l_1 <+ l_2 \rightarrow \exists l, l_2 \sim l_1 ++ l$

Fold / Sum

Perm.foldl_eq foldl ignores order if f is commutative+associative
 $l_1 \sim l_2 \rightarrow (\text{comm} + \text{assoc}) \rightarrow \text{foldl } f \ b \ l_1 = \text{foldl } f \ b \ l_2$

Perm.foldr_eq foldr ignores order if f is commutative+associative
 $l_1 \sim l_2 \rightarrow (\text{comm} + \text{assoc}) \rightarrow \text{foldr } f \ b \ l_1 = \text{foldr } f \ b \ l_2$

sum_nat Nat sum preserved
 $l_1 \sim l_2 \rightarrow l_1.\text{sum} = l_2.\text{sum}$

Perm.all_eq all f preserved
 $l_1 \sim l_2 \rightarrow l_1.\text{all } f = l_2.\text{all } f$

Perm.any_eq any f preserved
 $l_1 \sim l_2 \rightarrow l_1.\text{any } f = l_2.\text{any } f$

Flatten & Pmap

Perm.flatten flatten of outer list preserved
 $l_1 \sim l_2 \rightarrow l_1.\text{flatten} \sim l_2.\text{flatten}$

Perm.pmap proof-carrying map preserves $\sim$
 $l_1 \sim l_2 \rightarrow l_1.\text{pmap } f \ H_1 \sim l_2.\text{pmap } f \ H_2$

Proof tactics for `Perm` goals — useful combinations in practice:

- `exact Perm.refl _ / rfl` — trivial perm
- `exact p.symm` — flip direction
- `exact p₁.trans p₂` — chain permutations
- `exact perm_middle` — move element from middle to front
- `exact p.mem_iff.mp h` — transfer membership
- `exact p.mem_iff.mpr h` — transfer membership (other direction)
- `rwa [p.mem_iff]` — rewrite goal using perm membership
- `exact perm_cons_append_cons a p` — insert `a` into middle